Logic Specification Template 論理仕様テンプレート

Student 学生	三留慎太郎	**Date** 日付	2026/04/09
Program プログラム	7	**Program #** プログラム番号	
Instructor インストラクタ	日下部茂	**Language** 言語	Java

Design References 設計参照

Parameters パラメータ

String filename データを保持する csv ファイル名

double xk

Main
String fileName にファイル名を代入
double xk に見積もり値代入
linkedList にファイルからデータを格納したリストを代入
double[] stdDevs = Calculator.calcStandardDevition(linkedList)で x,y の標準偏差計算
double Beta0 = Calclator.calcBeta0(linkedList)でベータ 0 計算
double Beta1 = Calclator.calcBeta1(llinkedList)でベータ 1 計算
double yk = Beta0 + (xk * Beta1)で予測値を計算
double rxy = Beta1 * stdDevs[0] / stdDevs[1]で相関係数計算
double r2 = rxy * rxy
double tailArea = Calclator.calcTailArea(double rxy, int dataNum)
double Range = Calclator.calcRange(LinkedListManager<Pair<Doubke, Double>>linkedList, int dataNum)
double UPI = yk + Range
double LPI = yk - Range
Output.output(rxy, r2, tailArea, Beta0, Beta1, yk, Range, UPI, LPI)で出力

Logic Specification Template 論理仕様テンプレート

Student 学生	三留慎太郎	**Date** 日付	2026/04/09
Program プログラム	7	**Program #** プログラム番号	
Instructor インストラクタ	日下部茂	**Language** 言語	Java

Design References 設計参照

Parameters パラメータ

calcTailArea(double rxy, int dataNum)
double upperLimitX = (|rxy| * √(n-2)) / √(1-(rxy * rxy))積分範囲の上限を計算する
double p = calcSimpsonIntegral(double upperLimitX, int dof)t 分布を 0 から上限まで数値積分した値を計算
tailArea = 1 – 2p テイルエリアを計算する
return tailArea
calcRange(LinkedListManager<Pair<Double, Double>>linkedList, int dataNum, double xk)
tFunctionValue = findUpperLimit (n-2, 0.35)
sigmaValue = CalcStandardDeviation(LinkedListManager<Pair<Double, Double>>linkedList, int dataNum)
averages[] = calcAverage(linkedList)
double denominator
while(List 全て処理)
denominator += (xi – aerages[0])^2
Range = tFunctionValue * sigmaValue * √(1 + (1/n) + ((xk – average[0]) / denominator))
return Range

Logic Specification Template Instructions
論理仕様テンプレートの説明

Purpose	- To contain the pseudocode for a program, component, or system - To enable precise and complete program implementation - To facilitate thorough design and implementation reviews and inspections - - プログラム、コンポーネント、またはシステムの擬似コードを記述するため - 正確かつ完全なプログラム実装を可能にするため - 徹底的な設計および実装のレビューと検査を容易にするため
General	- Use this template to document the program's detailed logic. - After implementation and testing, update the template to reflect the actual implemented product. - During detailed design, write the pseudocode needed to describe all of the program's logic. - Use plain language and avoid using programming instructions wherever practical. - - このテンプレートを使用して、プログラムの詳細なロジックを文書化する - 実装とテストの後、実装された実際の製品を反映するようにテンプレートを更新 - 詳細設計段階では、プログラムのすべてのロジックを記述するために必要な擬似コードを記述する - 平易な言葉を使用し可能な限りプログラミングインストラクションの使用は避ける
Header	- Enter your name and the date. - Enter the program name and number. - Enter the instructor's name and the programming language you are using. - - 名前と日付を入力する - プログラム名と番号を入力する - インストラクタ名と使用するプログラミング言語を入力する
Design References	List the references used to produce the program's logical design. - the Operational, Functional, and State templates - the program's requirements - any other pertinent source - - プログラムの論理設計を作成するために使用した情報源をリストアップする - - 操作テンプレート、機能テンプレート、および状態テンプレート - - プログラムの要求 - - その他の関連情報源
Parameters	- Where needed, define any parameters or abbreviations used. - Avoid duplicating definitions on other templates and reference these other definitions where they are needed. - - 必要に応じて、使用したパラメータまたは略語を定義する - 他のテンプレートとの定義の重複を避け、必要に応じてこれらの定義を参照する